Task and Project Management System

Idea, requirements, and architecture for the beta. Later milestones follow this document.

1. Project Idea

The system helps a small team run its work through projects and tasks. A project manager creates a project and adds tasks, each with a title and a due date, then assigns each task to one member. The assigned member moves the task from **To Do** to **In Progress** to **Done**, members can leave comments on it, and each member can open a list of only the tasks assigned to them. The manager reviews the project on a board grouped by status. Members are stored with the project and used as assignees. Because login is out of scope, the user picks which project member they are acting as, and each member has a role of manager or member.

Objective. Deliver a beta that completes this path from end to end, with the business rules kept apart from the screens so the same behavior can be tested directly and through the application.

Expected outcome. One application in which a manager can create a project, add and assign tasks, and read the board, and a member can update status, comment, and see their own tasks. The six functional requirements below are the beta. Later milestones keep this architecture and extend the same system.

Justification. The domain is familiar and the rules are explicit: a required title, a fixed status order, a single assignee, one board, and a personal task list. That scope is concrete enough to construct and test, and small enough for a team of six to finish without an external API or a large data model.

2. Core User Stories

Stories describe the scenarios the beta must support. Login and sign-up are out of scope. Each story is the source of the functional requirement with the same number.

ID	Statement
US1	As a project manager, I want to create a project and add tasks with a title and a due date so that the team has a clear list of work.
US2	As a project manager, I want to assign a task to a member so that ownership of the work is explicit.
US3	As a team member, I want to move my task from To Do to In Progress to Done so that progress is visible.
US4	As a team member, I want to comment on a task so that decisions stay attached to the work.
US5	As a project manager, I want to see a project’s tasks grouped by status so that I can tell what is in progress and what is finished.
US6	As a team member, I want to see only the tasks assigned to me so that I can focus on my own work.

3. Functional Requirements

ID	Statement
FR1	A manager can create a project and add tasks. Each task has a title and a due date.
FR2	A manager can assign a task to one member of that project.
FR3	The member assigned to a task can change its status only in the order To Do, then In Progress, then Done.
FR4	A member can add a comment to a task. The comment keeps its author and the time it was added, and comments are shown in the order they were created.
FR5	The system shows every task in a project grouped by status.
FR6	A member can view the tasks assigned to them, ordered by due date.

4. Non-Functional Requirements

These constraints are part of the beta and will be checked by tests in later milestones.

ID	Quality	Statement
NFR1	Data integrity	A task submitted with an empty title is rejected with a validation error, nothing is stored, and every stored task belongs to exactly one project.
NFR2	Data integrity	A task can only be assigned to a member of its own project. Any other assignee is rejected and the task is not changed.
NFR3	Reliability	A status change that skips a step or moves backward is rejected, and the stored status stays unchanged.
NFR4	Auditability	Once a comment is saved, its author, text, and time cannot be changed.
NFR5	Performance	Listing up to 100 tasks in one project completes within one second on a local run.
NFR6	Privacy	A member’s task list never contains tasks assigned to another member.
NFR7	Testability	Every domain rule can be unit-tested on its own, without a database or user interface.

Traceability for later test design: US1–FR1–NFR1, US2–FR2–NFR2, US3–FR3–NFR3, US4–FR4–NFR4, US5–FR5–NFR5, US6–FR6–NFR6. NFR7 applies to all six.

5. System Architecture

Type. Layered architecture: one deployable application and one database.

Justification. The beta is a single flow over one set of projects, members, tasks, and comments. Four layers separate the screens, the use cases, the business rules, and storage. The interface depends on the application layer, and the application layer depends on the domain. The domain defines repository interfaces, and the persistence layer implements them, so the domain does not depend on the database or the screens. Domain rules can therefore be tested on their own, and the board flow can still be exercised end to end. This layering is the structure later milestones will follow.

The application layer calls the domain for rules and calls persistence, through the repository interfaces, for storage. Results move back up. Persistence never calls the domain; it only implements the domain's interfaces.

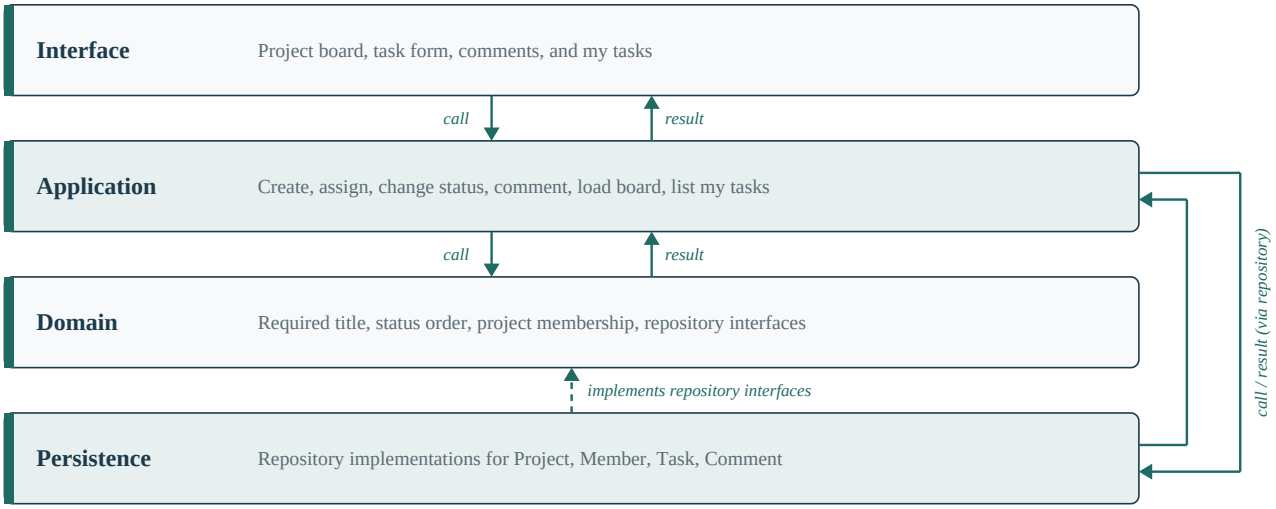

Figure 1. Layered components, the call / result flow, and the repository interfaces that keep the domain independent of storage.

Data flow — mark a task Done. The interface sends the requested status to the application service. The service loads the task through the task repository and asks the domain whether a task that is In Progress may become Done. When the rule allows it, the service saves the new status through the repository. The service reloads the project's tasks and the interface shows the task in the Done group. When the rule rejects the change, the service returns an error and persistence is not updated. Opening "my tasks" uses the same path: the service asks the repository for that member's tasks, the domain leaves the status rules untouched, and the interface shows them in due-date order.

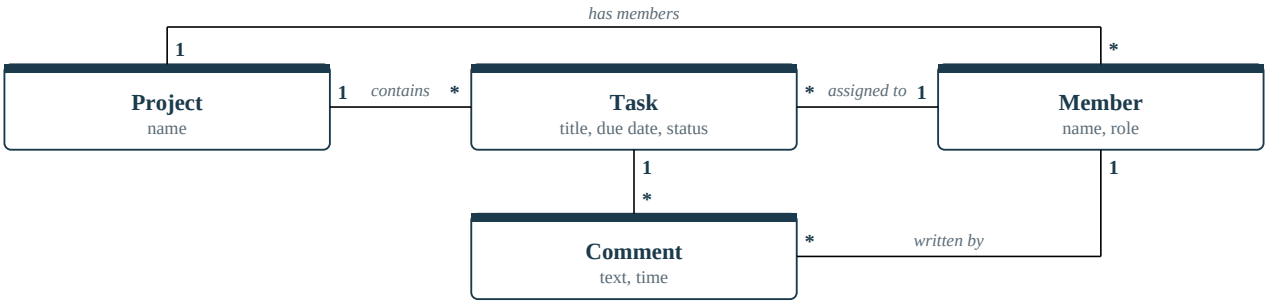

Figure 2. Beta data model. A project contains many tasks and many members. A task has one assignee and many comments, and each comment has one author.

Carried into later milestones. Build FR1–FR6 inside these four layers. Test them against US1–US6, and check NFR1–NFR7 with the same cases. Keep this layering; the beta grows by extending these use cases, not by replacing the structure.